

End-to-End Object Detection with Transformers: Fine-tuning DETR on Custom Datasets

Abstract—This paper presents an implementation and analysis of the Detection Transformer (DETR), a novel approach to object detection that eliminates the need for hand-designed components such as non-maximum suppression and anchor generation. We demonstrate the fine-tuning of DETR on a custom balloon dataset and provide comprehensive insights into its transformer-based architecture, bipartite matching loss mechanism, and object query learning paradigm. The model achieves competitive performance while maintaining architectural simplicity and end-to-end trainability.

Index Terms—Object detection, transformers, DETR, bipartite matching, computer vision, deep learning

I. INTRODUCTION

Object detection has traditionally relied on hand-crafted components including anchor boxes, region proposals, and non-maximum suppression procedures. These components encode explicit prior knowledge about the detection task but add complexity to the detection pipeline. The Detection Transformer (DETR), introduced by the Facebook Research team, reformulates object detection as a direct set prediction problem, streamlining the traditional pipeline through an end-to-end learnable architecture.

DETR demonstrates accuracy and runtime performance comparable to established methods like Faster R-CNN on the COCO object detection dataset while offering conceptual simplicity and requiring no specialized libraries. This work presents a detailed analysis of DETR's architecture and demonstrates its fine-tuning on a custom balloon detection dataset.

II. METHODOLOGY

A. Overall Architecture

The DETR model comprises three primary components: a convolutional neural network (CNN) backbone for feature extraction, a transformer encoder-decoder architecture for reasoning about object relationships, and prediction heads for generating final detection outputs. Fig. 1 illustrates the complete architecture.

The detection pipeline proceeds as follows: an input RGB image of dimensions $H_0 \times W_0 \times 3$ is processed through the CNN backbone to generate C lower-dimensional features with dimensions $H \times W$, where typically $C = 2048$, $W = W_0/32$, and $H = H_0/32$. For a $640 \times 640 \times 3$ input image, the backbone produces a $20 \times 20 \times 2048$ activation map.

B. CNN Backbone and Feature Processing

The backbone CNN generates spatially organized feature representations that must be reformatted for transformer processing. Each feature map is flattened into an $H \times W$ vector and concatenated to form a sequence suitable for the encoder. Spatial positional encodings, either learned or predefined, are added to these flattened convolutional features to preserve spatial information throughout the transformer processing.

C. Transformer Encoder

The encoder consists of N stacked encoder layers, each containing a multi-head self-attention (MHSA) mechanism, residual connections with layer normalization (Add & Norm), and feed-forward neural networks. The architecture closely follows the original transformer design with a key modification: spatial positional encodings are added exclusively to the Key and Query matrices rather than to all inputs.

D. Transformer Decoder and Object Queries

The decoder introduces a novel concept of object queries— N learnable positional embeddings (implemented as `nn.Parameter` in PyTorch) that serve as input to the decoder. These $N = 100$ learned embeddings are transformed through the decoder to produce bounding box predictions. Unlike traditional anchor boxes that encode size priors, object queries learn spatial location patterns.

The decoder processes these queries through multiple self-attention and encoder-decoder attention layers. The encoder output and spatial positional encoding are added to the key matrix before the multi-head attention layer. Through training, each object query specializes in detecting objects within specific image regions, with the attention mechanism enabling communication between queries to avoid redundant predictions.

Analysis of trained object query centroids reveals spatial specialization patterns, where different queries focus on distinct image regions such as bottom-left, center, or top-right areas. Importantly, these queries are class-agnostic and learn spatial patterns independent of object categories.

E. Prediction Heads

The prediction heads consist of two feed-forward networks operating on decoder outputs to generate class predictions and bounding box coordinates. Each of the N object queries produces one prediction tuple (c, b) containing a class label and bounding box. When fewer than N objects are present in an image, the remaining predictions output a special "no object" class.

F. Bipartite Matching Loss

DETR employs a bipartite matching loss to address the set prediction problem. For an image containing M objects ($M < N$), the ground truth is augmented with $(N - M)$ "no object" instances to match the fixed number of predictions. The challenge is establishing correspondence between predictions and ground truth, as there is no predetermined ordering.

The bipartite matching is computed using the Hungarian algorithm, which finds an optimal one-to-one assignment between predicted and ground truth objects that minimizes the total matching cost. This approach eliminates low-quality predictions and removes the need for non-maximum suppression.

The total loss combines classification and localization components:

$$L_{\text{total}} = L_{\text{class}} + L_{\text{bbox}}$$

where L_{class} is the negative log-likelihood for class predictions, and L_{bbox} is a linear combination of L1 loss and Generalized Intersection over Union (GIoU) loss. The GIoU component ensures scale-invariance across

small and large bounding boxes.

III. EXPERIMENTAL SETUP

A. Dataset

We evaluate DETR on a custom balloon detection dataset containing 61 training images and 13 validation images. The dataset originally provided annotations in VIA format, which were converted to COCO format using the VIA2COCO conversion tool to meet DETR's input requirements.

The dataset follows the standard COCO directory structure:

- annotations/custom_train.json
- annotations/custom_val.json
- train2017/ (training images)
- val2017/ (validation images)

B. Training Configuration

The model was fine-tuned for 200 epochs using a pretrained DETR backbone. Training was conducted following the standard DETR training procedure with appropriate learning rate scheduling and data augmentation techniques.

IV. RESULTS AND DISCUSSION

A. Training Metrics

Model performance was evaluated using multiple metrics throughout training:

Average Precision (AP): The primary COCO dataset metric showed consistent improvement over the training period, demonstrating the model's ability to accurately localize and classify balloon instances.

Loss Components: Training loss analysis revealed convergence patterns across multiple components including total loss, classification loss, L1 bounding box distance loss, and GIoU loss. The complementary nature of L1 and GIoU losses provided both absolute distance penalties and scale-invariant optimization.

Error Metrics: Cardinality error (prediction count accuracy) and class error monitored the model's ability to predict the correct number of objects and their classifications respectively.

B. Qualitative Results

Visual inspection of predictions on test data demonstrated DETR's capability to accurately detect balloon instances with precise bounding box localization. The model successfully handled varying balloon sizes, orientations, and partial occlusions while maintaining clean predictions without duplicate detections.

C. Key Advantages

The DETR architecture offers several advantages over traditional object detection methods:

1. **End-to-End Training:** All components are jointly optimized without requiring separate training stages.
2. **Elimination of Hand-Designed Components:** No need for anchor box engineering, region proposal networks, or non-maximum suppression.
3. **Set-Based Prediction:** The bipartite matching formulation naturally handles variable numbers of objects without predefined output structures.
4. **Global Reasoning:** The transformer architecture enables reasoning about relationships between all objects and the global image context simultaneously.
5. **Architectural Simplicity:** The model consists of standard components without specialized libraries or complex post-processing.

V. CONCLUSION

This work presented a comprehensive analysis of the Detection Transformer architecture and demonstrated its effectiveness through fine-tuning on a custom balloon detection dataset. DETR's reformulation of object detection as a set prediction problem with transformer-based processing offers a conceptually simpler alternative to traditional detection pipelines while maintaining competitive performance.

The key innovations—object queries, bipartite matching loss, and end-to-end transformer processing—provide a foundation for future object detection research. The architecture's simplicity and lack of hand-designed components make it particularly suitable for adaptation to new domains and tasks.

Future work could explore extensions to other detection tasks, investigation of efficiency improvements for real-time applications, and analysis of failure modes in challenging scenarios such as dense object arrangements or extreme scale variations.

REFERENCES

- [1] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," arXiv preprint arXiv:2005.12872, 2020.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," in Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
- [4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017.

[5] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in European Conference on Computer Vision, 2014, pp. 740–755.